Guía de GitFlow para el Proyecto HabanaEntregas

Ingeniería de Software

Resumen

Esta guía tiene como objetivo servir de documento de consulta y referencia para la aplicación del modelo de ramificación **GitFlow** en el desarrollo del proyecto **HabanaEntregas**, un sistema de control de medios (equipos electrónicos y mobiliario) para una empresa multisucursal. El documento está diseñado para complementar el taller práctico y proporcionar a los estudiantes una referencia estructurada, formal y de consulta continua durante las 8 semanas del proyecto. Se abordan los fundamentos de Git, el modelo GitFlow, el flujo de trabajo paso a paso, la integración continua y despliegue continuo (CI/CD), buenas prácticas, anti-patrones y el mapeo de los 4 hitos del proyecto a ramas GitFlow.

Índice

1. Introducción **5**

1.1. Justificación del uso de GitFlow en HabanaEntregas 5

1.2. Objetivos de la guía 5

2. Fundamentos de Git **6**

2.1. Las tres áreas de Git 6

2.2. Ramas en Git 6

2.3. Sincronización con el remoto 6

3. GitFlow: Modelo de Ramificación **7**

3.1. Ramas principales 7

3.1.1. `main` (o `master`) 7

3.1.2. `develop` 7

3.2. Ramas de apoyo 7

3.2.1. `feature/*` — Ramas de funcionalidad 7

3.2.2. `release/*` — Ramas de preparación de release 8

3.2.3. `hotfix/*` — Ramas de corrección urgente 8

3.3. Diagrama de flujo 8

4. Flujo de Trabajo Paso a Paso **9**

4.1. Configuración inicial del repositorio 9

4.2. Desarrollo de una feature 10

4.3. Pull Request y merge 10

4.4. Creación de una release 11

4.5. Hotfix en producción 12

5. Integración Continua y Despliegue Continuo (CI/CD) **13**

5.1. Diferencias conceptuales 13

5.2. Pipeline típico para HabanaEntregas 13

5.3. Configuración de entornos 14

6. Buenas Prácticas **14**

6.1. Convenciones de nomenclatura 14

6.2. Conventional Commits 14

6.3. Commits atómicos 15

6.4. Branch protection 16

6.5. Documentación continua 16

7. Anti-patrones (Errores a Evitar) **16**

8. Mapeo de Hitos a Ramas GitFlow **17**

8.1. Hito 1 (Semanas 1-2): Configuración inicial, LDAP, CRUD básico . . . 17

8.2. Hito 2 (Semanas 3-4): CRUD completo + reportes (a)(b) → Entrega E1 18

8.3. Hito 3 (Semanas 5-6): Reportes (c)(d) + PDF → Entrega E2 18

8.4. Hito 4 (Semanas 7-8): Integración intranet + IIS → Entrega E3 19

9. Bibliografía y Recursos **19**

9.1. Referencias obligatorias 19

9.2. Referencias complementarias 19

10. Práctica opcional para la Semana **20**

1. Introducción

GitFlow es un modelo de organización de ramas Git propuesto por Vincent Driessen en 2010 que define un conjunto de reglas estrictas para la creación, uso y fusión de ramas, con el objetivo de gestionar proyectos que tienen releases planificadas y ciclos de entrega formales. Aunque en años recientes ha ganado popularidad el enfoque **Trunk-Based Development** para proyectos con despliegue continuo, GitFlow sigue siendo especialmente adecuado para contextos donde existen entregas formales, auditoría y validación por parte del cliente.

1.1. Justificación del uso de GitFlow en HabanaEntregas

El caso de estudio HabanaEntregas presenta las siguientes características que hacen de GitFlow la estrategia adecuada:

- **Ciclo de releases planificado:** 4 hitos con fechas definidas (H1-H4) en 8 semanas.
- **Entregables formales:** 3 entregas (E1, E2, E3) con acta de aceptación firmada por el cliente.
- **Necesidad de trazabilidad:** Cada entrega debe ser reproducible y auditable.
- **Validación UAT (User Acceptance Testing):** El cliente exige pruebas formales antes de cada entrega.
- **Equipo de 5 estudiantes con roles rotativos:** Requiere un flujo de trabajo claro y documentado.

1.2. Objetivos de la guía

Al finalizar la lectura y aplicación de esta guía, el estudiante será capaz de:

1. Comprender el modelo de ramificación GitFlow y su relación con los 4 hitos de HabanaEntregas.
2. Diferenciar CI, CD (Continuous Delivery vs Continuous Deployment) y pipeline.
3. Aplicar ramas `feature/`, `develop`, `release/`, `hotfix/` y `main` al caso de estudio.
4. Diseñar un pipeline CI/CD para el stack ASP.NET Core MVC + EF Core + SQL Server.
5. Gestionar los riesgos R1-R8 del proyecto usando GitFlow como mitigación.

2. Fundamentos de Git

Antes de adentrarnos en GitFlow, es fundamental comprender los conceptos básicos de Git que sustentan todo el modelo.

2.1. Las tres áreas de Git

Git organiza el trabajo en tres áreas principales:

Cuadro 1: Las tres áreas de Git

Área	Descripción	Comando típico
Working Directory	Archivos tal como los ves en tu editor. Modificaciones sin trackear.	—
Staging Area (Index)	"Sala de espera" del próximo commit. Seleccionas qué cambios entran.	`git add <archivo>`
Repositorio Local (.git)	Base de datos de commits. Historia completa del proyecto.	`git commit -m "mensaje"`

El flujo básico es: **Working Directory** → (`git add`) → **Staging Area** → (`git commit`) → **Repositorio Local** → (`git push`) → **Repositorio Remoto**.

2.2. Ramas en Git

Una rama es un **puntero ligero y barato** a un commit. En Git, crear una rama no copia archivos ni duplica el directorio de trabajo; simplemente crea un nuevo puntero a un commit existente. Esto hace que el trabajo con ramas sea extremadamente eficiente.

2.3. Sincronización con el remoto

Cuadro 2: Comandos de sincronización con el remoto

Comando	Propósito
`git clone <url>`	Copiar un repositorio remoto a local
`git remote add origin <url>`	Vincular repositorio local con remoto
`git push -u origin <rama>`	Subir rama al remoto y establecer tracking
`git pull`	Traer cambios del remoto y fusionarlos

3. GitFlow: Modelo de Ramificación

3.1. Ramas principales

GitFlow define **dos ramas principales** que existen permanentemente durante todo el ciclo de vida del proyecto:

3.1.1. `main` (o `master`)

- **Propósito:** Almacena el historial oficial de releases.
- **Características:** Solo recibe código mediante merges desde ramas `release/` o `hotfix/`. Cada commit en `main` debe estar etiquetado con un número de versión (tag).
- **En HabanaEntregas:** Contiene las versiones entregadas al cliente (v1.0-E1, v1.1-E2, v2.0-E3).

3.1.2. `develop`

- **Propósito:** Rama de integración para features en desarrollo.
- **Características:** Contiene el historial completo del proyecto, incluyendo features en progreso. Es la rama base para nuevas features.
- **En HabanaEntregas:** Recibe todas las features completadas y aprobadas mediante Pull Request.

3.2. Ramas de apoyo

GitFlow define **tres tipos de ramas temporales** que se crean y destruyen según sea necesario:

3.2.1. `feature/*` — Ramas de funcionalidad

- **Origen:** `develop`
- **Destino:** `develop` (mediante merge o squash)
- **Convención de nombres:** `feature/<descripción-corta>`
- **Ejemplo:** `feature/ldap-auth`, `feature/crud-sucursal`
- **Regla:** Las features **nunca** interactúan directamente con `main`.

3.2.2. `release/*` — Ramas de preparación de release

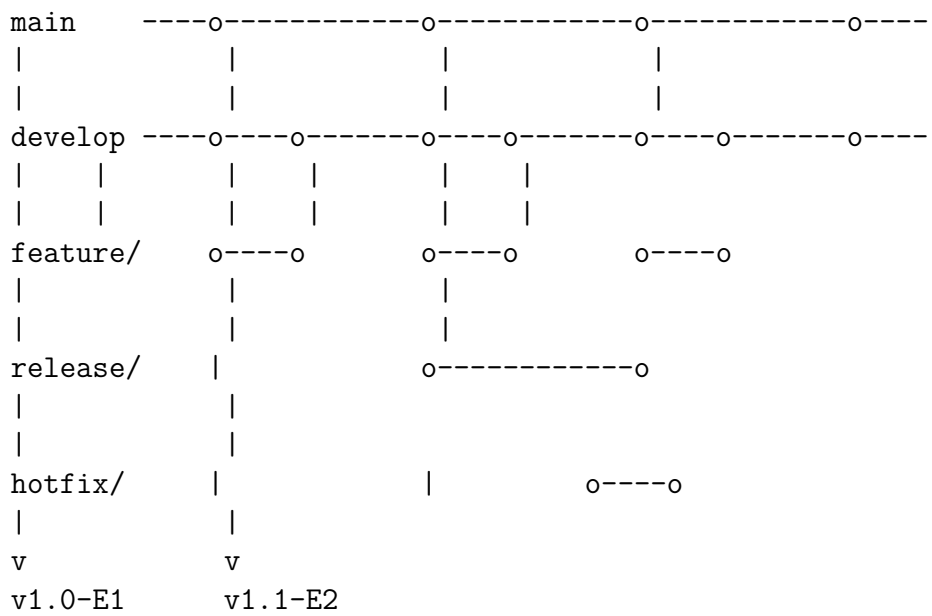
- **Origen:** `develop`
- **Destino:** `main` (con tag) **y** `develop` (back-merge)
- **Convención de nombres:** `release/<versión>`
- **Ejemplo:** `release/1.0`, `release/1.1`
- **Regla:** Una vez creada la rama `release/`, **no se pueden añadir nuevas features**. Solo se permiten correcciones de bugs, documentación y tareas de preparación del release.

3.2.3. `hotfix/*` — Ramas de corrección urgente

- **Origen:** `main` (**NO desde `develop`**)
- **Destino:** `main` (con tag) **y** `develop` (back-merge)
- **Convención de nombres:** `hotfix/<descripción>`
- **Ejemplo:** `hotfix/pdf-layout`
- **Regla:** Un hotfix nace de `main` porque `develop` puede contener features inestables que no deben incluirse en la corrección urgente.

3.3. Diagrama de flujo

El flujo completo de GitFlow en HabanaEntregas es el siguiente:



Cadena de trazabilidad: `feature/` → `develop` → `release/` → `tag` → `main` → producción (IIS).

4. Flujo de Trabajo Paso a Paso

4.1. Configuración inicial del repositorio

Paso 1: Crear repositorio en GitHub/Azure DevOps.

Paso 2: Clonar a local:

```
    git clone <url>
    cd habanaentregas

```

Paso 3: Crear ramas base:

```
    git checkout -b main
    git checkout -b develop
    git push -u origin main
    git push -u origin develop

```

Paso 4: Configurar `.gitignore` para .NET:

```
    echo 'bin/
    obj/
    *.user
    .vs/' > .gitignore
    git add .gitignore
    git commit -m 'chore: add .gitignore'
    git push origin develop

```

Paso 5: Activar **branch protection** en GitHub:

- Settings → Branches → Add rule
- **Branch name pattern:** `main`, `develop`
- **Require a pull request before merging** ✓
- **Require status checks to pass before merging** ✓
- **Require approvals:** 1
- **Require review from Code Owners** ✓

Paso 6: Definir `CODEOWNERS` para revisión obligatoria:

```
# .github/CODEOWNERS
*       @lider-tecnico @qa-lead
/Services/  @dev-backend
/Data/      @dev-backend
/Web/       @dev-frontend
/Tests/     @qa-lead

```

4.2. Desarrollo de una feature

Paso 1: Crear rama feature desde `develop` (**NO desde `main`**):

```
git checkout develop
git pull origin develop
git checkout -b feature/ldap-auth

```

Paso 2: Implementar la funcionalidad.

Paso 3: Hacer commits atómicos con **Conventional Commits**:

```
git commit -m "feat(auth): integrar LDAP con Novell.Directory.Ldap
"
git commit -m "test(auth): anadir tests unitarios LdapAuthService"
git commit -m "docs(auth): documentar configuracion LDAP en README
"

```

Paso 4: Empujar la feature al remoto:

```
git push -u origin feature/ldap-auth

```

Paso 5: Abrir **Pull Request** hacia `develop`.

4.3. Pull Request y merge

Flujo de PR:

1. **Abrir PR** `feature/*` → `develop`
2. **Revisión por pares:** QA/DevOps + Líder técnico
3. **CI verde obligatorio:** El pipeline debe pasar
4. **Merge squash** a `develop`

Checklist de revisión por pares:

☐ ¿Pasa tests xUnit con cobertura ≥60 %?

☐ ¿Sigue arquitectura 3 capas (Controller→Service→Repository)?

☐ ¿No rompe navegación (≤3 niveles)?

☐ ¿Documenta la API pública con comentarios XML?

☐ ¿No introduce secretos en el código (LDAP_HOST, etc.)?

☐ ¿Usa Conventional Commits en el mensaje de merge?

☐ ¿No hay `console.log` ni código comentado?

☐ ¿Actualiza el `CHANGELOG.md` si aplica?

Merge squash:

```
    gh pr merge 14 --squash --delete-branch

```

El merge squash colapsa todos los commits de la feature en **un solo commit limpio** en `develop`, preservando el histórico dentro del PR pero manteniendo la rama principal legible.

4.4. Creación de una release

Paso 1: Congelar feature-list (cerrar nuevas features).

Paso 2: Abrir `release/1.0` desde `develop`:

```
    git checkout -b release/1.0 develop

```

Paso 3: Bugfixes de última hora solo en `release/`:

```
    git commit -m 'fix(reportes): corregir ordenamiento por marca'

```

Paso 4: Merge a `main` con tag anotado:

```
    git checkout main
    git merge --no-ff release/1.0 -m 'Merge release/1.0 into main'
    git tag -a v1.0-E1 -m 'Entrega 1: CRUD + reportes (a)(b)'
    git push origin main --tags

```

Paso 5: Merge **también** a `develop` (back-merge):

```
    git checkout develop
    git merge --no-ff release/1.0 -m 'Back-merge release/1.0 into
   develop'
    git push origin develop

```

Paso 6: Generar artifact con `dotnet publish`:

```
    dotnet publish -c Release -o ./publish
    zip -r v1.0-E1.zip ./publish

```

Paso 7: Publicar artifact en GitHub Releases:

```
    gh release create v1.0-E1 --title "Entrega E1" v1.0-E1.zip

```

Versionado semántico: `MAJOR.MINOR.PATCH` — `v1.0-E1` = primera entrega del cliente (versión 1.0).

4.5. Hotfix en producción

Protocolo de hotfix:

1. **Detectar bug crítico** en producción (E2 ya entregado).
2. **Abrir `hotfix/` desde `main` (NO desde `develop`):**

```
      git checkout main
      git checkout -b hotfix/pdf-layout main

```

3. **Corregir y commit:**

```
      git commit -m 'fix(pdf): corregir encabezado en QuestPDF'

```

4. **Merge a `main` con tag patch:**

```
      git checkout main
      git merge --no-ff hotfix/pdf-layout -m 'Merge hotfix/pdf-
   layout into main'
      git tag -a v1.1.1 -m 'Hotfix PDF: encabezado corregido'
      git push origin main --tags

```

5. **Merge TAMBIÉN a `develop`:**

```
      git checkout develop
      git merge --no-ff hotfix/pdf-layout -m 'Back-merge hotfix/pdf
   -layout into develop'
      git push origin develop

```

6. **Publicar nuevo artifact y desplegar a producción.**
7. **Smoke tests** en producción.

Regla de oro: Hotfix nace de `main`, no de `develop` — `develop` puede tener features inestables.

5. Integración Continua y Despliegue Continuo (CI/CD)

5.1. Diferencias conceptuales

Cuadro 3: Diferencias entre CI, Continuous Delivery y Continuous Deployment

Concepto	Definición	En HabanaEntregas
CI (Integración Continua)	Integrar código varias veces al día con build y tests automáticos	Cada push a `feature/*` dispara build + tests xUnit
Continuous Delivery	Cada cambio que pasa CI está **listo** para desplegar, pero requiere aprobación manual	**SÍ se usa:** entregas E1-E3 con aprobación del cliente y UAT
Continuous Deployment	Cada cambio que pasa CI se despliega **automáticamente** a producción	**NO se usa:** el cliente exige UAT formal por hito

5.2. Pipeline típico para HabanaEntregas

Etapas del pipeline:

1. **Checkout:** `actions/checkout@v4`
2. **Restore:** `dotnet restore`
3. **Build:** `dotnet build -c Release`
4. **Test:** `dotnet test -logger trx -collect:'XPlat Code Coverage'`
5. **Publish:** `dotnet publish -c Release -o artifact`
6. **Deploy:** IIS via webdeploy o run-from-package

YAML mínimo (GitHub Actions):

```yaml
name: CI
on:
push:
branches: [develop, feature/*]
jobs:
build:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- name: Setup .NET
uses: actions/setup-dotnet@v4
with:
dotnet-version: '8.0.x'
- name: Restore
```

```
15      run: dotnet restore
16      - name: Build
17      run: dotnet build -c Release --no-restore
18      - name: Test
19      run: dotnet test --no-build --logger trx
20
```

5.3. Configuración de entornos

Cuadro 4: Configuración de entornos

Entorno	Rama origen	Trigger	Aprobación	Propósito
Dev	`develop`	push	No	Integración continua
Staging	`release/*`	merge a release	No	UAT interna
Prod	`main` + tag	tag `v*`	Sí (manual)	Cliente final

Secretos: `LDAP_HOST`, `LDAP_PORT`, `LDAP_BASE_DN`, `SQL_CONNECTION_STRING`, `INTRANET_API_KEY`. Nunca en el repositorio; siempre en GitHub/Azure Secrets.

Estrategia de despliegue: Blue-green en IIS (dos sitios: `prod-actual`, `prod-nuevo`). Switch tras smoke tests. Rollback: re-deploy del artifact anterior en <5 min.

Smoke tests post-deploy: login LDAP + CRUD Sucursal + 1 reporte.

6. Buenas Prácticas

6.1. Convenciones de nomenclatura

Cuadro 5: Convenciones de nomenclatura de ramas

Tipo de rama	Convención	Ejemplo
Feature	`feature/<descripción-corta>`	`feature/ldap-auth`
Release	`release/<versión>`	`release/1.0`
Hotfix	`hotfix/<descripción>`	`hotfix/pdf-layout`

6.2. Conventional Commits

La especificación **Conventional Commits 1.0** define una estructura clara para los mensajes de commit:

```
<tipo>[contexto opcional]: <descripcion>

[cuerpo opcional]

[footer(s) opcional(es)]

```

Tipos recomendados:

Cuadro 6: Tipos de Conventional Commits

Tipo	Propósito	Versión semántica
`feat`	Nueva funcionalidad	MINOR
`fix`	Corrección de bug	PATCH
`docs`	Documentación	—
`refactor`	Refactorización sin cambio funcional	—
`test`	Añadir o modificar tests	—
`chore`	Tareas de mantenimiento	—
`build`	Cambios en build o dependencias	—
`ci`	Cambios en configuración CI	—

Ejemplos:

```
feat(auth): integrar LDAP con Novell.Directory.Ldap
fix(pdf): corregir encabezado en QuestPDF
docs(readme): documentar configuracion del pipeline
refactor(repository): extraer interfaz IRepository
test(services): anadir tests para ReporteService

```

BREAKING CHANGE: Se indica con `!` después del tipo o con un footer `BREAKING CHANGE:`:

```
feat(api)!: eliminar endpoint obsoleto /api/v1/medios

```

6.3. Commits atómicos

Regla: Un commit = un cambio lógico. Si necesitas "y" en el mensaje, probablemente son dos commits.

Ejemplo correcto:

```
 git commit -m "feat(auth): integrar LDAP con Novell.Directory.Ldap
"
 git commit -m "test(auth): anadir tests unitarios LdapAuthService"
 git commit -m "docs(auth): documentar configuracion LDAP en README
"

```

Ejemplo incorrecto:

```
1     git commit -m "integrar LDAP y anadir tests y documentar"  #
    Demasiado grande
2
```

6.4. Branch protection

GitHub permite configurar reglas de protección de ramas que impiden push directo y requieren PR aprobado con CI verde:

- **Require a pull request before merging:** Obliga a crear PR.
- **Require approvals:** Número mínimo de revisores.
- **Require status checks to pass:** CI debe estar verde.
- **Require review from Code Owners:** Revisión obligatoria de los dueños del código.
- **Dismiss stale pull request approvals:** Si se añaden nuevos commits, las aprobaciones previas se descartan.

6.5. Documentación continua

- **README.md:** Instrucciones de instalación, ejecución y despliegue.
- **CHANGELOG.md:** Registro de cambios por versión (siguiendo Keep a Changelog).
- **Wiki del repositorio:** Documentación técnica, decisiones de arquitectura, guías de configuración.
- **Comentarios XML:** Documentar API pública con `/// <summary>`.

7. Anti-patrones (Errores a Evitar)

Cuadro 7: Anti-patrones en GitFlow

Anti-patrón	Consecuencia	Corrección
Commit directo a `main`	Rompe protección, sin revisión	Solo PR aprobado + CI verde
Hotfix desde `develop`	Arrastra features inestables	Hotfix siempre desde `main`

Anti-patrón	Consecuencia	Corrección
Release sin tag	Sin trazabilidad de entrega	Tag anotado + artifact en Releases
PR gigante (>500 líneas)	Revisión superficial	PRs pequeños (<300 líneas)
Merge sin `-no-ff`	Pierde histórico de la rama	Usar `-no-ff` en release/hotfix
Secretos en repositorio	Riesgo de seguridad	Secrets en GitHub/Azure + `.gitignore`
Rama `feature/` de >1 semana	Conflictos de merge	Features cortas (<3 días)
Commits sin Conventional Commits	Historial ilegible	Usar prefijos `feat:`, `fix:`, etc.
No hacer back-merge a `develop`	`develop` queda desactualizado	Siempre merge a `main` y `develop`
No configurar branch protection	GitFlow es solo una sugerencia	Activar reglas en GitHub

8. Mapeo de Hitos a Ramas GitFlow

8.1. Hito 1 (Semanas 1-2): Configuración inicial, LDAP, CRUD básico

Cuadro 8: Mapeo del Hito 1

Aspecto	Detalle
Tareas	Configurar proyecto, LDAP, BD, Repository, entidades, login, CRUD, migración
Ramas	3 features → `develop` (squash)
Comandos clave	`git checkout -b feature/ldap-auth develop`
Criterios	Compila + login LDAP válido + rol Admin accede CRUD
Tag	`v0.1-dev` (release interna, sin entregable formal)

8.2. Hito 2 (Semanas 3-4): CRUD completo + reportes (a)(b) → Entrega E1

Cuadro 9: Mapeo del Hito 2

Aspecto	Detalle
Tareas	CRUD completo, vistas por tipo, ReporteService, Chart.js, tests ≥60 %
Ramas	3 features → `develop` → `release/1.0` → `main` + `develop`
Comandos clave	`git checkout -b release/1.0 develop`
Criterios	CRUD completo + reportes a/b + validaciones + cobertura ≥60 %
Tag	`v1.0-E1`
Artifact	`dotnet publish` → zip → GitHub Release

8.3. Hito 3 (Semanas 5-6): Reportes (c)(d) + PDF → Entrega E2

Cuadro 10: Mapeo del Hito 3

Aspecto	Detalle
Tareas	Reportes c/d, DTOs, PdfExportService, IReportExporter, tests PDF
Ramas	2 features + `release/1.1` + `hotfix/pdf-layout`
Comandos clave	`git checkout -b hotfix/pdf-layout main`
Criterios	Reportes c/d + PDF operativo + arquitectura extensible
Tag	`v1.1-E2`
Hotfix	`v1.1.1` (corrección urgente)

8.4. Hito 4 (Semanas 7-8): Integración intranet + IIS → Entrega E3

Cuadro 11: Mapeo del Hito 4

Aspecto	Detalle
Tareas	Contrato intranet, IntranetService, DashboardController, UAT, IIS
Ramas	2 features → `release/2.0` → `main` + tag
Comandos clave	`git checkout -b release/2.0 develop`
Criterios	Intranet recibe datos + doc completa + IIS+SQL2017
Tag	`v2.0-E3`
Entrega	Acta firmada + transición a mantenimiento + soporte 3 meses

9. Bibliografía y Recursos

9.1. Referencias obligatorias

1. **Driessen, V. (2010).** *A successful Git branching model*. nvie.com. — Artículo original de GitFlow.

2. **Guía Metodológica HabanaEntregas.** Secciones 3, 4, 9.7, 10, 11. — Documento base del curso.

3. **Chacon, S., & Straub, B. (2014).** *Pro Git* (2nd ed.). Apress. Disponible en https://git-scm.com/book.

4. **Humble, J., & Farley, D. (2010).** *Continuous Delivery*. Addison-Wesley. Cap. 1-3.

9.2. Referencias complementarias

- **Conventional Commits 1.0.0.** https://www.conventionalcommits.org/.
- **Atlassian. Gitflow Workflow.** https://www.atlassian.com/git/tutorials.
- **GitHub Docs. About protected branches.** https://docs.github.com.
- **GitHub Docs. Understanding GitHub Actions.** https://docs.github.com.
- **Microsoft Learn. Azure Pipelines.** https://learn.microsoft.com.
- **Oh Shit, Git!?!** https://ohshitgit.com/ — Recetas para deshacer errores.

10. Práctica opcional para la Semana

1. Crear repositorio GitHub para el proyecto HabanaEntregas o el proyecto final.
2. Configurar `main` + `develop` + branch protection.
3. Abrir `feature/setup-proyecto`.
4. Hacer 3 commits con Conventional Commits.
5. Abrir PR y mergear con squash.
6. Activar GitHub Actions con el YAML mínimo del pipeline.
7. Leer Driessen (2010) — GitFlow original.
8. Empezar Humble & Farley cap. 1-3.

Referencias

[1] Driessen, V. (2010). *A successful Git branching model*. Recuperado de https://nvie.com/posts/a-successful-git-branching-model/

[2] Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. Recuperado de https://git-scm.com/book

[3] Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.

[4] Conventional Commits. (2020). *Conventional Commits 1.0.0*. Recuperado de https://www.conventionalcommits.org/

[5] GitHub. (2024). *About protected branches*. Recuperado de https://docs.github.com

[6] GitHub. (2024). *Understanding GitHub Actions*. Recuperado de https://docs.github.com

[7] Atlassian. (2024). *Gitflow Workflow*. Recuperado de https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow

[8] Universidad Placentia. (2024). *Guía Metodológica: Solución al Ejercicio HabanaEntregas*. Documento base de la asignatura Ingeniería de Software.